Charting Decision Guide

Use this guide to design how clinicians document care in your application. Work through encounter types, Visit Templates, structured data, signing, and template ownership to decide what the chart captures and how that information becomes usable FHIR resources.

Companion to the Charting docs (SOAP notes, diagnoses, vitals, allergies), plus Provider Visits, Structured Data Capture, PlanDefinition $apply, and QuestionnaireResponse $extract.

For each clinical workflow, make five decisions:

1. **Define visit types and their contents.** Decide what each encounter requires and how Visit Templates organize it.
2. **Choose how structured the chart should be.** Balance narrative documentation with fields needed for clinical workflows, reporting, and reuse.
3. **Choose how form responses become queryable data.** Define extraction and the resources it creates or updates.
4. **Define signing requirements.** Decide what is signed, who can sign or co-sign, and how later changes are handled.
5. **Assign Visit Template ownership.** Decide who can edit templates and how changes are governed.

Use the clinical context below to guide these choices. Treat the options as starting points, assign an owner to unresolved questions, and follow the linked technical docs for implementation.

Before you start

What care setting and specialty?

- Primary care
- Urgent care
- Specialty (cardiology, dermatology, orthopedics, etc.)
- Behavioral health
- Multi-specialty / mixed

Why it matters: care settings have different visit templates and charting needs. Behavioral health leans on standardized screening instruments; primary care emphasizes vitals and preventive care; specialty care often runs on protocol-driven order sets. Shapes every Visit Template in Visit types.

What types of encounters will be charted?

- In-person visits
- Telehealth visits
- Async / messaging-based encounters
- Group visits
- Some combination

Why it matters: encounter types may need distinct Visit Templates. Async encounters chart differently — the Subjective section may come from a messaging thread. Group visits raise the question of one note per patient vs. a shared note.

What are clinicians used to today, and what do they want to keep consistent?

- What EHR or charting tool are clinicians coming from?
- What note formats or workflows do they want to preserve?
- What gaps drove the move?

Why it matters: charting changes have high adoption risk. Knowing what clinicians expect to stay the same is as important as knowing what to improve.

What visit types do you provide, and what does each entail?

The recommended baseline is one Visit Template (PlanDefinition) per distinct visit type, combining structured forms (Questionnaires) with embedded order actions (ActivityDefinitions). `$apply` generates the CarePlan and Tasks at visit start.

⚠ **Visit types are the bridge to scheduling.** Define the mapping between charting Visit Templates and bookable HealthcareService offerings. Scheduling operations use explicit HealthcareService references, not code matching alone. See the Scheduling Decision Guide.

Questions:

- What are the distinct visit types? (annual wellness, acute sick, behavioral health intake, post-op follow-up, specialty consult, etc.)
- For each visit type, what does the clinician do during the encounter?
    - Measurements (vitals — BP, HR, weight, BMI, temp, SpO2)
    - Screening instruments (PHQ-9, GAD-7, AUDIT, fall risk, pediatric screens)
    - Orders (labs, imaging, referrals, prescriptions)
    - Narrative documentation
    - Review of patient history (problem list, allergies, meds)

What the answers drive:

For each visit type, the elements map to PlanDefinition components:

Visit element	How it's modeled in the PlanDefinition
Vitals	Questionnaire action; parses responses to Observations (`category = vital-signs`, LOINC codes)
Screening instruments (PHQ-9, GAD-7, etc.)	Questionnaires per each standard form

Visit element	How it's modeled in the PlanDefinition
Labs, imaging, and referrals	ActivityDefinition with `kind: ServiceRequest`; `$apply` creates the ServiceRequest and associated Task
Prescriptions	Create MedicationRequest through the prescribing workflow; `$apply` does not generate MedicationRequest resources
Narrative documentation	`ClinicalImpression.note` (captures the 'Assessment' part of SOAP)
Patient history review	Surfaced via the patient sidebar (see Chart structure), not modeled in the PlanDefinition itself

If visit types share most of their structure, factor common pieces into shared Questionnaires / ActivityDefinitions referenced by multiple PlanDefinitions.

Execution boundary: `$apply` processes top-level actions; it does not execute nested actions or evaluate `action.condition` as a guard. Use application or Bot logic to select and release conditional steps. Supported ServiceRequest `ActivityDefinition.dynamicValue` mappings can populate generated fields, but do not provide conditional workflow execution. Verify the generated resources before using a template for clinical work.

How structured should the chart be?

Medplum Recommendation: Structured wherever possible. Medplum's documented SOAP note approach maps each section to discrete resources — Observations for findings, ClinicalImpression for assessment, ServiceRequest / MedicationRequest / CarePlan for the plan. Reserve free-text (`ClinicalImpression.note`) for the assessment (A) component of SOAP.

⚠ **Recommended pattern.** Surface a patient sidebar (problem list, allergies, active medications) alongside the active note, with inline edit.

Questions:

- For each visit type from Visit types, what mix of structured forms vs. free-text does the workflow require?
- Are orders placed inline within the chart, or in a separate orders area?

What the answers drive:

Decision	Medplum Recommendation	Alternative (and tradeoff)
Structured vs. freeform	Structured forms; free-text only where clinicians require it	Heavy free-text. Tradeoff: not queryable; does not leverage FHIR data advantages
Order placement	Inline in the Visit Template, generated as Tasks	Separate orders module. Tradeoff: loses inline ordering context

How will form responses be parsed into queryable data?

Medplum Recommendation: always parse — without parsing, form data isn't queryable for analytics, alerting, or dashboards. The decision is *how*, driven by parsing complexity.

Questions:

- Does parsing need complex conditional behavior (score thresholds, external calls, multi-resource creation)?

What the answers drive:

Situation	Approach
Logic is relatively simple	**SDC + `$extract`.** Annotate the Questionnaire with SDC extensions; invoke `$extract` on submission. Less flexible, but extraction logic lives in one place with the form

Situation	Approach
Logic is complex	**Subscription + Bot.** Subscribe to QuestionnaireResponse and parse in code. More flexible (conditional logic, external calls), but Bot code changes are required when the form changes

What are the signing requirements?

The standard sign-off pattern marks `ClinicalImpression.status = completed` and creates Provenance targeting the Encounter to record the signer and timestamp. Define which content and resource versions the sign-off covers; an Encounter reference alone does not freeze the visit chart. Signing, locking, co-signing, and amendments need explicit application rules.

Questions:

- Which resource versions or document snapshot make up the signed content?
- Is co-signing required (attending co-signs resident, provider co-signs scribe)?
- Should signing lock immediately, or leave a post-sign editable window?
- Will amendments be possible / needed after a chart is locked?
- Is a queryable audit trail of every signing event required?

What the answers drive:

Situation	Approach
Single signer	ClinicalImpression `completed` plus Provenance recording attribution; identify the signed resource versions or retained document snapshot
Co-signature	Append additional Provenance entries per co-signer; surfaces multi-signer state. Typically a custom build

Situation	Approach
Lock timing	Choose immediate locking or an editable window; enforce the chosen write restrictions at the API. Creating Provenance does not lock resources
Post-lock amendments	Retain the original signed content and record an addendum, for example a DocumentReference linked to the Encounter; define how it references and supplements the original
Audit trail	Provenance per signing event; query via `Provenance?target=Encounter/{id}`

Who owns and edits Visit Templates?

Default: engineering implements template changes from a spec maintained by clinical operations — clinical ops owns *what* should change, engineering implements via API or migration scripts. Works well when changes are infrequent or warrant review.

If clinical ops needs to edit directly, the Medplum App provides a builder interface out of the box. It's relatively bare-bones (FHIR-shaped), so customers wanting a more visual or user-friendly experience for non-technical users can build a custom admin UI on top.

Questions:

- Will engineering implement template changes from a clinical-ops spec, or does clinical ops need to edit directly?

What the answers drive:

Situation	Approach
Engineering implements from a spec (default)	API or migration scripts; treated as code-reviewed artifacts

Situation	Approach
Clinical ops edits directly	Use the Medplum App's builder interface out of the box; build a custom admin UI on top if a more visual / user-friendly experience is required